

Кросплатформне програмування

07

Спадкування



Спадкування

Про що йтиметься:

- Що таке спадкування
- Суперклас і створення підкласу
- Конструктор підкласу
- Рівні доступу
- Переозначення методів
- Віртуальність методів і конструкторів
- Змінна суперкласу



Що таке спадкування

- Спадкування - один із базових механізмів ООП
- При спадкуванні один клас створюється на основі іншого класу
- Клас, на основі якого створюється новий клас, називається суперкласом
- Клас, що створюється на основі суперкласу, називається підкласом
- Підклас автоматично отримує всі відкриті поля та методи із суперкласу
- Підклас може мати лише один суперклас (заборона множинного спадкування)
- Підклас може бути суперкласом для іншого класу (багаторівневе спадкування)
- Клас може бути суперкласом відразу для декількох класів



Реалізація спадкування

- В описі підкласу після його імені через ключове слово `extends` вказується ім'я суперкласу
- У тілі підкласу описуються "додаткові" (у порівнянні з суперкласом) члени

Схема опису підкласу

```
class Підклас extends Суперклас{  
    // Поля і методи підкласу  
}
```

Наприклад

```
class Bravo extends Alpha{  
    // Додаткові (крім членів суперкласу Alpha)  
    // поля і методи підкласу Bravo  
}
```



Реалізація спадкування - приклад

Суперклас та підклас

```
// Описання суперкласу:
class Alpha{
    // Цілочислове поле:
    int number;
    // Метод:
    void show(){
        System.out.println("Числове поле: "+number);
    }
}
// Описання підкласу:
class Bravo extends Alpha{
    // Символьне поле:
    char symbol;
}
```

Об'єкт класу Bravo має:

поле	number
поле	symbol
метод	show()



Конструктор підкласу

Існують спеціальні правила опису конструктора підкласу:

- Під час створення об'єкта підкласу спочатку викликається конструктор суперкласу. Для цього вказується ключове слово **super**, після якого у круглих дужках передаються аргументи для конструктора суперкласу
- Якщо аргументів немає, круглі дужки (порожні) все одно вказуються
- У тілі конструктора підкласу інструкція з ключовим словом **super** має бути першою
- Якщо в тілі конструктора підкласу інструкція з ключовим словом **super** відсутня, автоматично викликається конструктор суперкласу без аргументів

Конструктор підкласу - приклад

```
class Alpha{           // Суперклас
    int number;         // Поле
    Alpha(int n){       // Конструктор
        number=n;
        System.out.println("number="+number) ;
    }
}
class Bravo extends Alpha{ // Підклас
    char symbol;         // Поле
    Bravo(int n,char s){ // Конструктор
        super(n);        // Виклик конструктора суперкласу
        symbol=s;
        System.out.println("symbol="+symbol) ;
    }
}
class Demo{             // Головний клас
    public static void main(String[] args){
        Bravo B=new Bravo(100,'B') ;
    }
}
```

Результат програми:

number=100

symbol=B

Пакети

- Класи групуються за пакетами, які відіграють роль своєрідних "контейнерів", або об'єднань класів (та інших пакетів, які зазвичай називають підпакетами)
- В межах контейнеру назва класу має бути унікальною, але якщо класи знаходяться в різних пакетах, то їх назви можуть співпадати

Щоб створити пакет, необхідно на початку програмного коду розмістити ключове слово `package` і назву створюваного пакету:

```
package назва_пакету;
```

Наприклад:

Для створення пакету `mydata` використовуємо інструкцію

```
package mydata;
```

При створенні підпакета назва пакета та підпакета розділяються крапкою:

```
package назва_пакета.назва_підпакета;
```

Наприклад:

Для створення підпакета `mybox` в пакеті `mydata` використовуємо інструкцію

```
package mydata.mybox;
```

Файл може містити лише одну `package`-інструкцію, і вона має бути першою інструкцією у файлі



Імпорт пакетів

- Щоб у програмі можна було використати клас чи класи з деякого пакета, клас або весь пакет слід імпортувати
- Для імпорту класу або пакету використовують інструкцію `import`, після якої вказується назва пакету і, через крапку, назва імпортованого класу або зірочка `*` (якщо імпортуються всі `public`-класи з пакету)

Наприклад, для імпорту класу з назвою `MyClass` з пакету `mydata` використовуємо інструкцію

```
import mydata.MyClass;
```

Наприклад, щоб імпортувати всі `public`-класи з пакету `mydata`, замість назви класу слід вказати зірочку `*`

```
import mydata.*;
```

Наприклад, щоб імпортувати класи з підпакета `mybox` пакета `mydata` використовуємо інструкцію

```
import mydata.mybox.*;
```

Файл може містити декілька `import`-інструкцій. Такі інструкції розміщуються на початку програмного коду, а якщо в програмному коді є `package`-інструкція, то відразу після неї.



Статичний імпорт

- При роботі зі статичними членами класів існує можливість спростити процедуру звернення до таких членів. Для цього використовують статичний імпорт
- При статичному імпорті після інструкції `import` вказується ключове слово `static`, а далі повне посилання на статичний член класу.
- Для імпорту всіх статичних членів класу замість імені члена класу використовують зірочку `*`.

Приклад інструкції статичного імпорту:

```
import static javax.swing.JOptionPane.showMessageDialog;
```

Приклад імпорту всіх статичних членів класу `JOptionPane`:

```
import static javax.swing.JOptionPane.*;
```



Рівні доступу

Місце, з якого отримуюємо доступ до члена класу	Специфікатор рівня доступу, з яким описаний член класу			
	public	protected	не вказаний	private
В тому ж класі	+	+	+	+
В підкласі в тому ж пакеті	+	+	+	-
В звичайному класі в тому ж пакеті	+	+	+	-
В підкласі іншого пакету	+	+	-	-
В звичайному класі в іншому пакеті	+	-	-	-

- Класи можуть описуватися зі специфікатором доступу `public`
- Якщо клас описаний без специфікатора доступу `public`, то він доступний у межах пакету
- Якщо клас описаний зі специфікатором доступу `public`, то він доступний і поза пакетом. Такий клас називають `public`-класом або відкритим класом
- В файлі може бути лише один відкритий клас, а назва файлу повинна співпадати з назвою відкритого класу



Спадкування і закриті члени - приклад

```
class Alpha{                                // Суперклас
    private int number;                     // Закрите поле
    void set(int n){
        number=n;
    }
    void show(){
        System.out.println("number="+number);
    }
    Alpha(int n){                           // Конструктор
        set(n);
        show();
    }
}
class Bravo extends Alpha{                 // Підклас
    Bravo(int n){                           // Конструктор
        super(n);                          // Виклик конструктора суперкласу
    }
}
class Demo{                                // Головний клас
    public static void main(String[] args){
        Bravo B=new Bravo(100);
        B.set(200);
        B.show();
    }
}
```

Результат програми:

number=100
number=200

Переозначення методів

- Методи, які успадковуються із суперкласу в підкласі можна переозначати
- Щоб переозначити успадкований метод, в підкласі даний метод описується наново у явному вигляді
- Не слід плутати переозначення і перевантаження методів

Існує ключове слово `final`, яке може використовуватись у наступний спосіб:

- Якщо описати змінну з ключовим словом `final`, то така змінна буде фактично константою - її значення змінити не можна (заборона зміни значення).
- Якщо описати з ключовим словом `final` метод, то такий метод не можна переозначати (заборона переозначення).
- Якщо описати з ключовим словом `final` клас, то на основі такого класу не можна буде створити підклас (заборона спадкування).



Переозначення методів - приклад

```
class Alpha{                                // Суперклас
    int number;                             // Поле
    void show(){                            // Метод
        System.out.println("number="+number);
    }
    Alpha(int n){                           // Конструктор
        number=n;
    }
}
class Bravo extends Alpha{                 // Підклас
    char symbol;                            // Поле
    Bravo(int n,char s){                   // Конструктор
        super(n);                          // Виклик конструктора суперкласу
        symbol=s;
    }
    void show(){                           // Переозначення методу
        super.show();
        System.out.println("symbol="+symbol);
    }
}
class Demo{                               // Головний клас
    public static void main(String[] args){
        Alpha A=new Alpha(100);
        Bravo B=new Bravo(200,'B');
        A.show(); // Виклик методу
        B.show(); // Виклик методу
    }
}
```

Результат програми:

number=100
number=200
symbol=B



Віртуальність методів

```
// Суперклас:
class Alpha{
    void show(){
        display();
    }
    void display(){
        System.out.println("Клас Alpha");
    }
}
// Підклас:
class Bravo extends Alpha{
    void display(){
        System.out.println("Клас Bravo");
    }
}
// Головний клас:
class Demo{
    public static void main(String[] args){
        Alpha A=new Alpha();
        Bravo B=new Bravo();
        A.show();
        B.show();
    }
}
```

Результат програми:

Клас Alpha

Клас Bravo



Змінна суперкласу

```
class Alpha{                                // Суперклас
    void show(){
        System.out.println("Клас Alpha");
    }
}
class Bravo extends Alpha{                // Підклас
    void show(){
        System.out.println("Клас Bravo");
    }
}
class Charlie extends Alpha{              // Підклас
    void show(){
        System.out.println("Клас Charlie");
    }
}
class Demo{                                // Головний клас
    public static void main(String[] args){
        Alpha A=new Alpha();
        A.show();
        A=new Bravo();
        A.show();
        A=new Charlie();
        A.show();
    }
}
```

Результат програми:

Клас Alpha
Клас Bravo
Клас Charlie

- Об'єктна змінна суперкласу може посилатись на об'єкт підкласу
- Доступ через об'єктну змінну суперкласу є тільки до тих полів і методів, які оголошені в суперкласі

Дякую за увагу!

Запитання?